

Document number: P2633R0
Date: 2022-08-15
Reply-to: Justin Cooke <jgc@cems.de>
Target audiences: EWG, SG1

thread_local_inherit : Enhancing thread-local storage

Abstract: To introduce a new storage class `thread_local_inherit`, which differs from `thread_local` in that variables with `thread_local_inherit` storage are initialized with the value held by the calling thread's instance of the variable. Only trivially copyable types may have `thread_local_inherit` storage. Their special initialization occurs during static initialization of the child thread, before the thread creation utility returns to the calling thread.

Motivation: `thread_local` variables are initialized *de novo* on thread creation, without regard to the values they held in the calling thread. However, there are often data that the child thread would like to inherit from the caller, but thenceforth maintain separate instances of. While it is possible to pass such data to a thread in arguments to its top-level function, the required information is typically not readily available in the context where the caller creates the thread. Indeed, the function creating the thread (or the programmer writing it) may not know of the existence of the data nor that the child thread needs them. Other methods of passing data to threads are cumbersome and likewise require the programmer to know what data need to be passed. There is merit in a language solution that supplies the required data automatically to the called thread, at the point and in the scope where they are required.

Description: Only variables of a trivially copyable type can be declared with the proposed new storage class specifier `thread_local_inherit`. The effect of the `thread_local_inherit` specifier is equivalent to the effect of `thread_local` except with respect to initialization. In the main thread only, a variable declared with `thread_local_inherit` is statically and dynamically initialized according to the same rules as for `thread_local` variables. During creation of a child thread, each `thread_local_inherit` variable is statically initialized to a trivial copy of the calling thread's instance of the variable. A `thread_local_inherit` variable will be dynamically initialized in the child thread only if it is not already completely initialized. A `thread_local_inherit` variable is considered completely initialized when it has been dynamically initialized, or when it has been statically initialized to a trivial copy of a completely initialized instance of the variable (*note*: intentionally recursive definition). If it is not already completely initialized, a `thread_local_inherit` variable is dynamically initialized in the child thread according to the same rules as for `thread_local` variables. The effect of these rules is that when a `thread_local_inherit` variable is first used during a thread's execution, there will have been exactly one dynamic initialization in the chain of copies leading to the current instance of the variable. To prevent data races, all static initialization in a child thread is sequenced before the thread's constructor returns to the parent thread.

The destructor of a `thread_local_inherit` object will be called on thread termination if the object is completely initialized, i.e. when it has either itself been dynamically initialized or has been statically initialized with a copy of a completely initialized instance of the variable.

Note: In the existing standard, 28.3.2 [cfenv.thread] in effect specifies that the floating point environment, considered as a variable, behaves as if it were declared with the proposed `thread_local_inherit` storage specifier.

Effect on existing code: The token `thread_local_inherit` becomes unavailable as an identifier.

Wording: (edits to N4910)

Table 5: Keywords
[tab:lex.key]

`thread_local_inherit`

6.7.4 Indeterminate values

[basic.indet]

¹ ...

[*Note 1* : Objects with static or thread storage duration are `statically` ~~zero~~-initialized, see 6.9.3.2. — *end note*]

6.7.5.3 Thread storage duration

[basic.stc.thread]

¹ All variables declared with the `thread_local` ~~or `thread_local_inherit`~~ keyword have thread storage duration. The storage for these entities lasts for the duration of the thread in which they are created. There is a distinct object or reference per thread, and use of the declared name refers to the entity associated with the current thread. ~~If a variable declared with the `thread_local_inherit` keyword is not of trivially copyable type (6.8.1), the program is ill-formed (diagnostic required).~~

6.9.3.2 Static initialization

[basic.start.static]

² Constant initialization is performed if a variable or temporary object with static or thread storage duration is constant-initialized (7.7). If constant initialization is not performed, a variable with static storage duration (6.7.5.2) or thread storage duration (6.7.5.3) is zero-initialized (9.4). ~~In threads other than the main thread (6.9.3.1), a variable declared with the `thread_local_inherit` specifier (6.7.5.3) is not constant- or zero-initialized, but instead is initialized as a trivial copy (6.8.1) of the calling thread's instance of that variable. Together, zero-initialization, ~~and~~ constant initialization ~~and trivial copy-initialization of variables declared with the `thread_local_inherit` specifier~~ are called *static initialization*; all other initialization is *dynamic initialization*. All static initialization strongly happens before (6.9.2.2) any dynamic initialization ~~in the same thread. All static initialization in a child thread happens strongly before the constructor of the thread (33.3.3) or `jthread` (33.3.4) object associated with the child thread returns to the calling thread).~~~~

6.9.3.3 Dynamic initialization of non-block variables

[basic.start.dynamic]

⁸ ~~The initialization of a variable is complete when its dynamic initialization is complete. Dynamic initialization in this sense includes initialization that was electively performed statically (6.9.3.2³). A variable declared with the `thread_local_inherit` storage specifier is also considered completely initialized if it has been statically initialized to a trivial copy of a completely initialized instance of the variable in the calling thread.~~

⁹ ~~Further dynamic initialization is not performed on a variable declared with the `thread_local_inherit` specifier (6.7.5.3) if it was already completely initialized by its static initialization (6.9.3.2).~~

8.8 Declaration statement

[stmt.dcl]

³ Dynamic initialization of a block variable with static storage duration (6.7.5.2) or thread storage duration (6.7.5.3) is performed the first time control passes through its declaration, **unless the variable is declared with the `thread_local inherit` specifier and was already completely initialized by its static initialization (6.9.3.2, 6.9.3.3).** ~~such a variable is considered initialized upon the completion of its initialization.~~

Additional edits:

`thread_local` à `thread_local` **or `thread_local inherit`** [9.2.2^{1,3,4}, 11.4.1¹², 11.4.9.3¹, 13.9.3³, 13.9.4²]

`thread_local` à `thread_local`, **`thread_local inherit`** [6.7.5.4¹, 9.1⁶, 9.3.4.1⁴,

and `thread_local` à `thread_local` **and `thread_local inherit`** [C.5.2³]

`thread_local` à `thread_local`
`thread_local inherit` [9.2.2¹, A.7.]

Implementation hints: In order to ensure that a block-scope variable with static or thread local storage is dynamically initialized exactly once, such a variable is typically implemented with an associated hidden status flag that is unset on static initialization and set on completion of dynamic initialization. Once the flag is set, the variable will not be dynamically initialized again. The flag is optional for a namespace-scope variable declared `thread_local` because it would not normally be exposed to multiple dynamic initialization attempts.

Regardless of whether it is declared in block or namespace scope, each `thread_local inherit` variable needs an associated status flag. The flag is unset during static initialization of the main thread. On child thread creation, all `thread_local inherit` variables and their status flags are statically initialized by trivial copy of the values held by the variables and their flags in the calling thread. Dynamic initialization of `thread_local inherit` variables follows the same rules as for `thread_local` variables, except that a variable whose status flag was already set by static initialization shall not be dynamically initialized.

Implementations typically place `thread_local` variables, along with any associated status flags, in a special segment, a new instance of which is created and constant-initialized on thread creation. `thread_local inherit` variables can likewise be implemented by placing them, with their associated flags, in a special part of this segment. The entire segment is constant-initialized in the usual way for the main thread. For a child thread, only the part-segment reserved for `thread_local` variables is constant-initialized in the usual way, while the part-segment reserved for `thread_local inherit` variables is initialized to a trivial copy of the calling thread's instance of the part-segment. The restriction that `thread_local inherit` variables be trivially copyable ensures that the part-segment can be initialized by a low-level bulk copy operation. The thread creator need only know the size of the part-segment, not the identity or offset of the individual variables within it. The copy operation should be completed before the thread creator returns to the calling thread.